

Conception et implémentation d'une application multitâche



Theo BONY – Saturnin WANONKOU

1- Introduction

L'objectif principal de ce projet est de :

- Concevoir des applications multitâche (non-temps réel) avec une approche dirigée par les événements
- Manipuler les mécanismes POSIX d'un système d'exploitation Linux
- Manipuler les mécanismes C11
- Concevoir une application multitâche avec une approche dirigée par le temps

2- Préambule

Dans ce préambule, nous avons découvert et identifier la manière de créer une tâche, un sémaphore et plus généralement les objets de synchronisation à l'aide des APIs POSIX.

a) Identifiez dans le programme le nom du thread.

Le nom du thread se nomme thread_1.

```
12 pthread_t thread_1;
```

b) Identifiez dans le programme le nom du sémaphore.

Le nom du sémaphore est semaphore.

```
13 sem_t *semaphore;
```

c) Identifiez dans le programme le nom du mutex.

Le nom du mutex est mutex.

```
15 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

d) Identifiez la création du thread.

La création du thread est réalisée avec l'API « pthread_create » à la ligne 44 du preambule.c.

```
44 pthread_create(&thread_1, NULL, produce, NULL);
```

e) Identifiez dans le programme le point d'entrée du thread.

Le point d'entrée du thread est « produce ».

```
44      pthread_create(&thread_1, NULL, produce, NULL);
```

f) Identifiez l'attente de la fin du thread pour terminer le processus courant.

Cette attente est réalisée par l'API « pthread_join » à la ligne 47 du préambule.c.

```
47      pthread_join(thread_1, NULL);
```

g) Identifiez le thread par défaut du processus courant.

Le thread par défaut du processus courant est le main() qui écrit à la ligne 33.

h) Identifiez les processus que vous allez mettre en jeu pour exécuter votre programme

Cette étape va varier selon la distribution de Linux utilisé mais lorsque l'on utilise Ubuntu. Il crée :

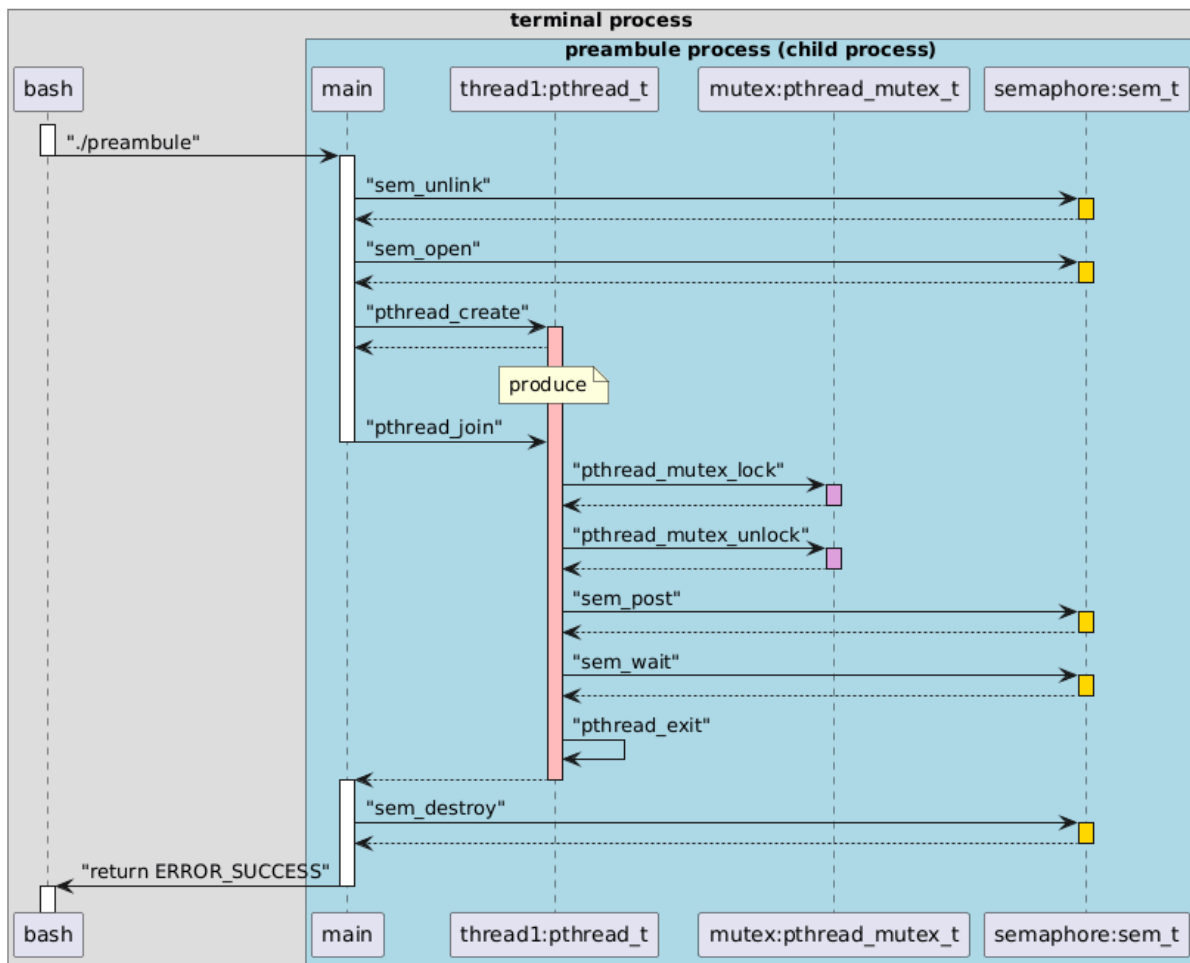
1 => gnome-terminal-server

2 => bash

3 => make

4 =>preambule

i) Diagramme UML de la conception de ce processus et de ce thread.



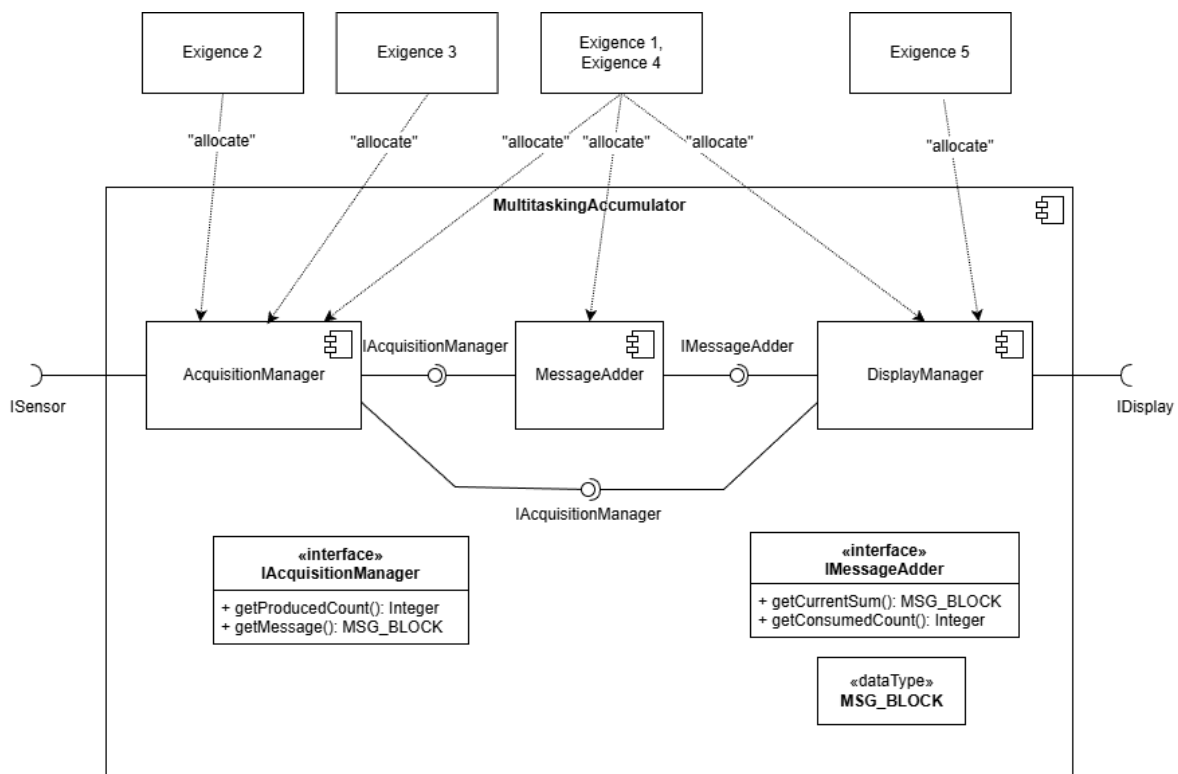
j) En partant de l'hypothèse qu'il n'existe pas d'exigences temporelles sur ce programme, sa conception est-elle complète ?

Même s'il n'existe pas d'exigences temporelles sur ce programme, on serait tenté de dire que la conception est complète. En revanche, il manque un point important qui est la gestion des erreurs. En effet, on ne gère pas les erreurs lors de la création et de la manipulation des ressources POSIX comme les threads, sémaphores et mutex. Les appels aux fonctions « pthread_create », « pthread_mutex_init » ou encore « sem_init » peuvent échouer. Donc non, la conception n'est pas complète.

3- Exercice-1 : MultitaskingAccumulator

L'objectif de cette partie est de développer un accumulateur multitâche qui gère l'acquisition en parallèle des données numériques de quatre sources asynchrones. Ces entrées viennent d'un module à part qui se nomme « SensorManager ». Le but est de les sommer et de produire deux sorties :

- Une sortie numérique correspondant à cette somme.
- Une sortie de diagnostic.



Dans notre cas, nous utilisons une approche par les événements ce qui est donc une approche asynchrone. En effet, les différents modules concernés réagissent dès que l'évènement se produit sans attendre ni bloquer l'exécution du programme.

La méthode « messageCheck » sert à vérifier la validité et l'intégrité des données reçues. C'est un failure detection et plus précisément un « Sanity Checking ». En d'autres termes, il sert à valider que les données acquises n'ont pas été corrompues ou modifiées pendant le transfert d'informations.

A ce stade de l'implémentation, on dénombre 1 processus qui est le process principal qui lance le main du fichier « MultitaskingAccumulator.c ». On observe ensuite 7 threads différents :

- Le premier est le thread qui lance le main().
- On a 4 threads qui représentent l'acquisition des données avec une par entrée (AcquisitionManager).
- Un thread pour l'addition et le cumul des valeurs représenté par le MessageAdder.
- Un thread pour l'affichage de la valeur et du diagnostic représenté par DisplayManager.

Pour revenir à la méthode « messageCheck », nous allons maintenant détailler comment est réalisé le checksum. Dans cette méthode, le checksum est obtenu en réalisant un XOR sur tous les éléments du tableau mData. On compare ensuite cette valeur avec le checksum qui est fourni dans la structure MSG_BLOCK.

```
unsigned int messageCheck(volatile MSG_BLOCK* mBlock) {
    unsigned int i, tcheck=0;
    for(i=0; i < DATA_SIZE; i++)
        tcheck ^= mBlock->mData[i];
    if(tcheck == mBlock->checksum) {
        printf("[OK      ] Checksum validated\n");
        return 1;
    } else {
        printf("[ FAILED] Checksum failed, message corrupted\n");
        return 0;
    }
}
```

On initialise la variable tcheck à 0. Ensuite, on vient parcourir tout le tableau mBlock et faire un XOR avec chaque élément. On compare ensuite tcheck et checksum contenu dans mBlock pour retourner 1 si le checksum est validé sinon 0.

Pour notre implémentation finale, nous sommes arrivés à une architecture logicielle détaillée, nous nous basons principalement sur un pattern de multiwrite/singleread pour remplir le buffer servant à stocker la donnée recueillie au niveau du capteur. Nous avons donc rajouté deux sémaphores : un de lecture et l'autre d'écriture, ainsi que 3 mutex dont 2 qui servent à la synchronisation des threads d'écriture pour le contrôle des index de deux tables. Ces tables permettent de gérer la disponibilité en lecture ou en écriture des cases de notre buffer de données. Le troisième mutex qu'en a lui permet de réaliser la synchronisation des opérations sur le compteur « produceCount ».

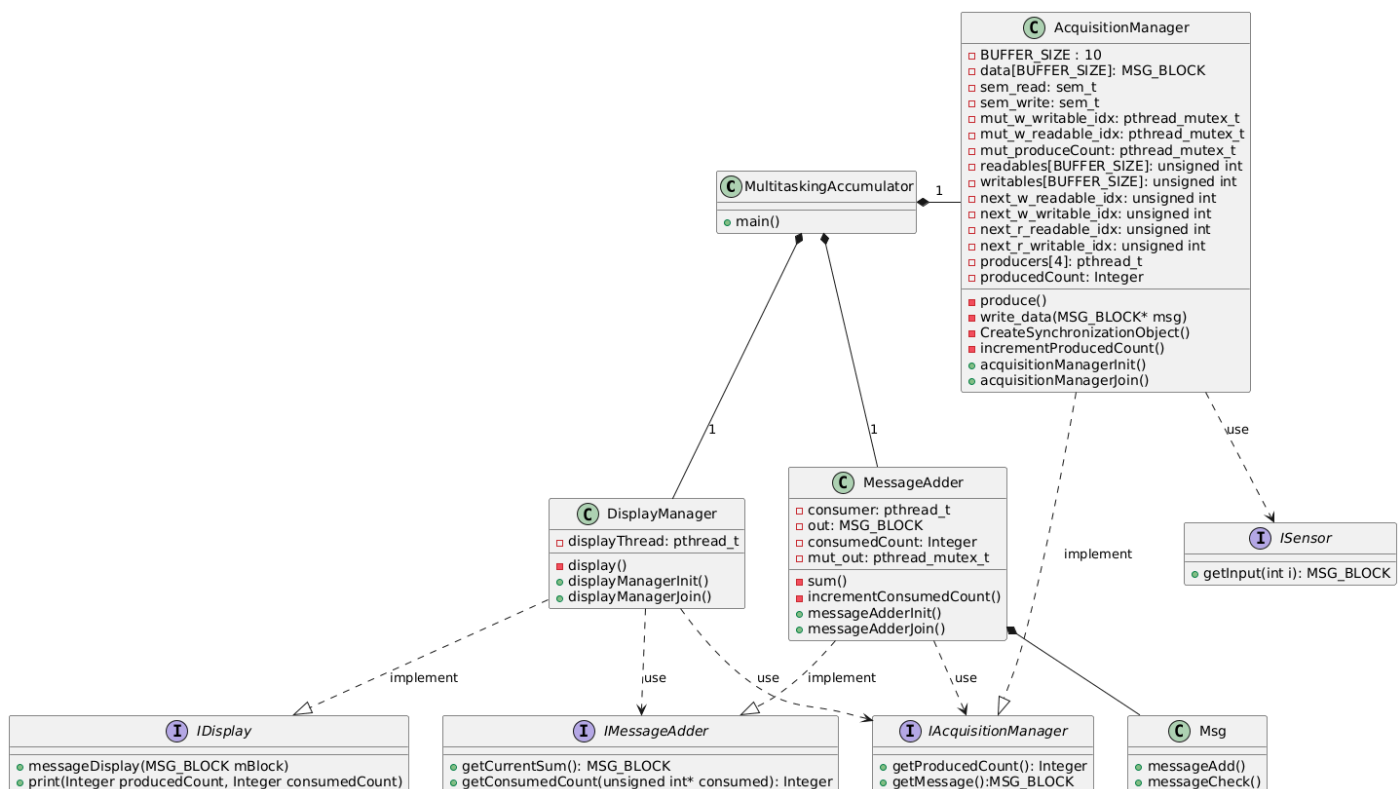


Figure 1: Architecture logicielle détaillée

En résumé du voici une explication du fonctionnement des thread d'écriture et de lecture:

Thread d'écriture:

- J'attends qu'une place écrivable soit prête : $P(\text{sem_write})$;
- Je récupère ponctuellement le mutex `mut_w_writable_idx` pour réserver ma case writable contenant l'indice d'une position où je peux écrire dans le buffer;
- J'effectue mon écriture;
- Je récupère ponctuellement le mutex `mut_w_readable_idx` pour réserver ma case readable

- J'y écris l'index de la case que je viens de remplir;
- Je signale une donnée disponible : V(sem_read)

Thread de lecture (unique) :

- J'attends qu'une place lisible soit prête : P(sem_read);
- Je récupère mon indice de lecture dans readable[next_r_readable_idx]
- Je lis ma donnée;
- J'écris dans writables[next_r_writable_idx] l'indice que j'ai récupéré;
- Je signale une case disponible en écriture : V(sem_write)

Sem_read est bien initialisé à 0 et sem_write à la taille du buffer

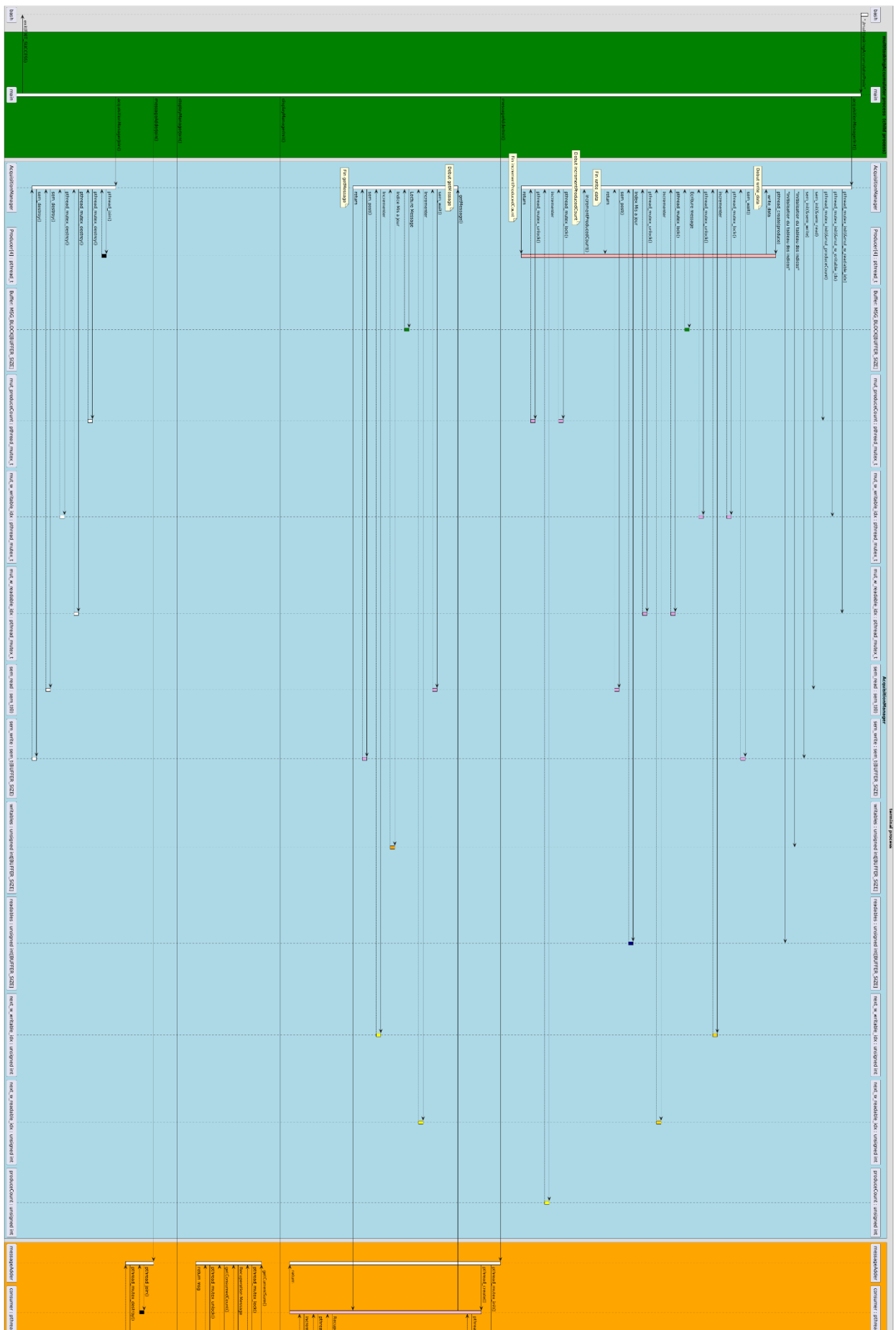


Figure 2: Diagramme de séquence du multitaskingAccumulator

[illegible]

- Les checksums sont bien validés avant écriture
- La concurrence entre threads de lecture et d'écriture est bien gérée
- Les messages sont bien affichés et cohérents

Cette méthode s'assure de relever la valeur de `consumedCount` pendant qu'il détient le mutex de contrôle de la sortie du `messageAdder` (`mut_out`) puis de l'assigner à la variable pointée si non null. Cela fonctionne à tout moment car l'incrément de ce compteur par le thread `adder` se fait toujours en possession du mutex `mut_out`.

10

4- Pour aller plus loin en conception

Pour réaliser ce TP et cet exercice, nous avons orienté l'architecture détaillé en utilisant des tâches. Mais nous aurions effectivement pu utiliser des processus POSIX.

L'utilisation de processus est particulièrement intéressant pour une conception orientée « sûreté de fonctionnement » car il permet de mettre en place un partitionnement spatiale ce qui permet de créer une mémoire séparée pour chaque donc pas de risques de modification ou d'écrasement de valeur. C'est un confinement total par rapport aux autres composants ce qui permet de conserver l'intégrité du code, des données et des ressources.

Mais cela est compliqué pour le travail en parallèle et le partage de donnée car il est très compliqué de faire communiquer des processus entre eux. Donc, dans le cas où nous utilisons le système de processus, nous aurions pu réutiliser les mutex et les sémaphores pour la synchronisation des processus entre eux mais l'architecture générale doit changer totalement car elle était basée sur le partage de variables globales.

Si on avait voulu mettre en place ce système processus et de communication, on aurait pu utiliser plusieurs solutions vues en cours :

- Mémoire partagée (ou fichier partagé) pour permettre aux processus de partager une partie de leurs espaces d'adresses virtuels
- Sémaphores nommés pour permettre aux processus de synchroniser leur exécution.
- Queue de messages pour envoyer des données formatées aux processus souhaités.
- Pipe, signals et sockets.

Comme vu précédemment, notre programme fonctionne, mais elle n'est pas très optimisée comme tenu de volume important d'appels systèmes qu'il génère via les API Posix.

Nous nous proposons de maintenant d'éviter en partie les API POSIX pour la synchronisation de nos compteurs tout au moins (en l'occurrence `produceCount` dont l'utilisation est intensive). Pour cela nous le déclarons comme variable `_Atomic` avec la ligne :

```
//producer count storage
_Atomic volatile unsigned int produceCount = 0;
```

Cela permet de rendre atomiques toutes les opérations d'écriture/lecture ayant lieu sur `produceCount`. Nous pouvons donc percevoir `produceCount++` comme atomique et

donc plus besoin d'un mutex de synchronisation entre threads. Nos méthodes deviennent simplifiées :

```
static void incrementProducedCount(void)
{
    //TODO
    ++produceCount;
}

unsigned int getProducedCount(void)
{
    return produceCount;
}
```

Et nos résultats obtenus sont tout aussi corrects :

[illegible]

Cette méthode fonctionne mais son comportement atomique n'est pas garantie pour des variables manipulables en dehors des threads (typiquement les variables contrôlées par des périphériques par exemple). Nous pouvons utiliser `atomic_compare_exchange_weak` dont le comportement atomique est garanti à l'intérieur du CPU. Voici ci-dessous les nouvelles implémentations de nos méthodes `increment` et `get`.

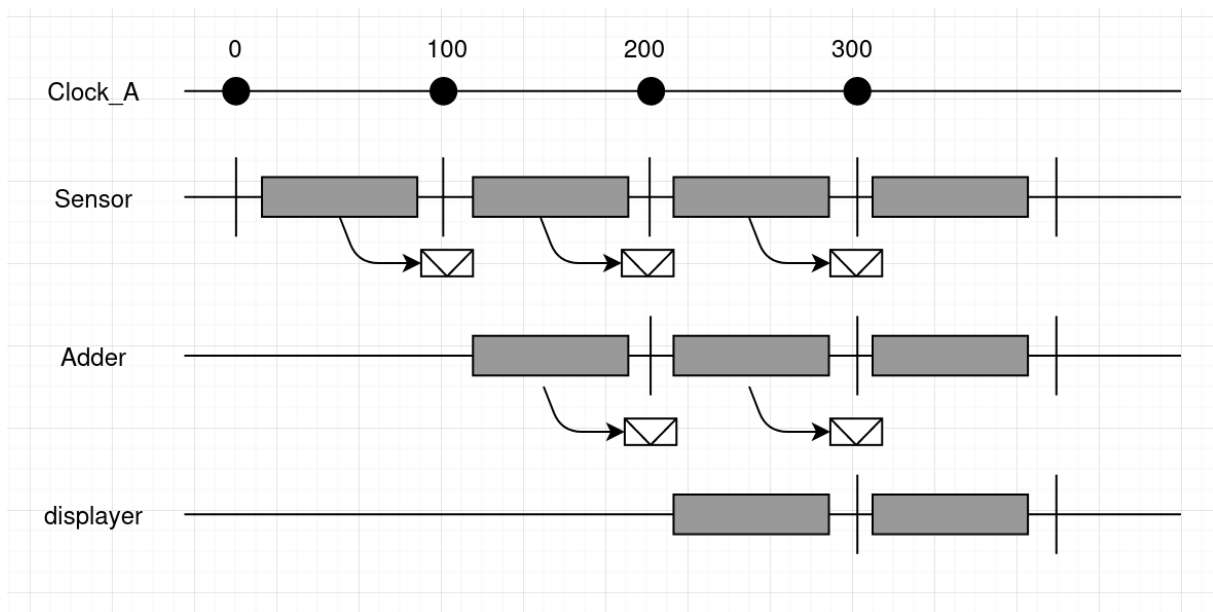
```
static void pCountLockTake(){
    unsigned int expected = 0;
    while(!atomic_compare_exchange_weak(&lock, &expected, 1))
        expected=0;
}

static void pCountLockRelease(){
    unsigned int expected = 1;
    while(!atomic_compare_exchange_weak(&lock, &expected, 0))
        expected=1;
}
```


version Posix car il n'utilise d'appels systèmes mais il reproduit le fonctionnement d'un mutex avec la méthode `atomic_compare_exchange_weak`.

5 - Conception détaillée en utilisant une approche dirigée par le temps

Maintenant si on veut utiliser une approche dirigée par le temps, on se retrouve avec une approche qui est synchrone. En effet, elle est reliée à une référence temporelle (une horloge) qui permet de coordonner l'exécution des différentes tâches. Toutes les exécutions des tâches sont décidées à la conception et sont bornées entre deux instants.



6 - Conclusion

Ce travail pratique nous a permis d'expérimenter la conception et l'implémentation d'une architecture multitâche en gérant le cas de plusieurs écrivains pour un seul lecteur. On a notamment pu utiliser les APIs POSIX. Nous avons aussi pu voir des alternatives à ces API comme Atomic. Et en dernière partie, nous avons réfléchi à une implémentation, cette fois-ci, dirigé par le temps.